Python for Robotics

Dr. TNOURJI, Abdellah
Research Scientist
The UM6P Vanguard Center

Autonomous Drone Engineering Internship

Mohammed VI Polytechnic University, UM6P , September 9, 2026

Outline

1 Python Refresher for Robotics

2 End-to-End Object-Oriented Python for Robotics

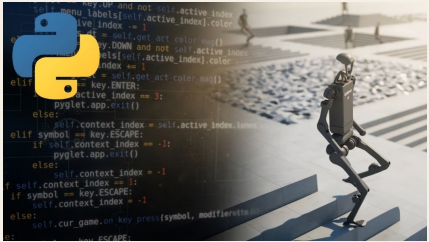
1 Python Refresher for Robotics
- Core Language Foundations
- Functions and Program Structure
- Object-Oriented Python for Robotics
- Errors, Files, and Configuration
- Python Environments and Dependencies
- NumPy and Basic Robotics Data Processing
- Logging, Debugging, and Integration

2 End-to-End Object-Oriented Python for Robotics

Introduction

A robotics program transforms inputs into useful outputs

Sensors and configuration → Python logic → telemetry, commands, logs, and detections



1 Python Refresher for Robotics
- Core Language Foundations
- Functions and Program Structure
- Object-Oriented Python for Robotics
- Errors, Files, and Configuration
- Python Environments and Dependencies
- NumPy and Basic Robotics Data Processing
- Logging, Debugging, and Integration

2 End-to-End Object-Oriented Python for Robotics

Variables and Data Types

Names that refer to values used by the robot

Common Types

- `int`: number of detections
- `float`: altitude in metres
- `bool`: mission enabled or disabled
- `str`: frame or object name
- `None`: value not available yet

```
altitude_m = 4.0
detection_count = 3
mission_active = True
target_label = "inspection_car"
latest_position = None
```

Robotics Habit

Include units in names such as `altitude_m`, `time_s`, and `rate_hz`. Python is dynamically typed, but engineering meaning must remain explicit.

Lists, Tuples, and Dictionaries

Choose a container according to the meaning of the data

Intuitive Roles

- **List**: ordered collection that may change
- **Tuple**: fixed ordered record
- **Dictionary**: named key-value lookup

```
ranges_m = [2.4, 2.1, 1.9]
position_ned = (3.0, -1.0, -4.0)
state = {
    "mode": "ORBIT",
    "battery_pct": 78.0
}
```

Selection Rule

Use a tuple for one position, a list for a history of measurements, and a dictionary when names such as `mode` or `battery_pct` make the record clearer.

Indexing, Slicing, and Unpacking

Access exactly the data needed by the algorithm

```
position_ned = (3.0, -1.0, -4.0)
north, east, down = position_ned

recent_ranges = [2.8, 2.5, 2.2, 2.0]
latest = recent_ranges[-1]
last_three = recent_ranges[-3:]
```

Index

Select one element. Python starts counting at zero; index −1 means the last element.

Unpacking

Give meaningful names to the components of a structured value.

Mutability and Copies

Two names may refer to the same changing object

```
measurements = [1.0, 2.0]
alias = measurements
alias.append(3.0)
# measurements is now [1.0, 2.0, 3.0]

independent_copy = measurements.copy()
```

Intuition

Unexpected shared changes can corrupt a measurement history, configuration, or message buffer. Use `copy()` when an independent list is required.

Conditions

Choose behavior from validated information

```
if latest_position is None:
    status = "waiting_for_odometry"
elif altitude_m > max_altitude_m:
    status = "limit_exceeded"
else:
    status = "ready"
```

Robotics Interpretation

Conditions implement decisions such as waiting for sensor data, rejecting invalid parameters, changing mission phase, or suppressing an unsafe output.

Boundary

An `if` statement is program logic, not proof of flight safety. Safety requires validated requirements, system-level safeguards, testing, and operational procedures.

For Loops

Apply the same operation to a known collection

```
detections = ["car", "person", "car"]
car_count = 0

for label in detections:
    if label == "car":
        car_count += 1
```

Robotics Uses

Process every detection, inspect each sensor sample, publish a sequence of waypoints, or declare a set of ROS 2 parameters.

Readable Iteration

Iterate directly over values. Use `enumerate(...)` only when an index is genuinely needed.

While Loops and Repeated Callbacks

Repeat until a condition changes, but always define an exit

```
attempt = 0
while attempt < 3:
    attempt += 1
    print(f"Connection attempt {attempt}")
```

Use `while` When

The number of repetitions depends on a changing condition.

In ROS 2

Timers and callbacks usually provide repetition. Avoid blocking loops inside callbacks.

Failure Mode

An unlimited loop can freeze a node or block message processing. Define a stop condition, timeout, or cancellation path.

1 Python Refresher for Robotics
- Core Language Foundations
- Functions and Program Structure
- Object-Oriented Python for Robotics
- Errors, Files, and Configuration
- Python Environments and Dependencies
- NumPy and Basic Robotics Data Processing
- Logging, Debugging, and Integration

2 End-to-End Object-Oriented Python for Robotics

Functions

Give one reusable operation a clear name

```
import math

def planar_distance_m(x1, y1, x2, y2):
    """Return 2D Euclidean distance in metres."""
    dx = x2 - x1
    dy = y2 - y1
    return math.hypot(dx, dy)

distance_m = planar_distance_m(0.0, 0.0, 3.0, 4.0)
```

Why Functions Matter

A function isolates one responsibility: calculate distance, validate parameters, convert coordinates, filter detections, or build a message.

Parameters, Returns, and Type Hints

Make the function contract visible

```
def clamp(value: float,
          minimum: float,
          maximum: float) -> float:
    return max(minimum, min(value, maximum))
```

Function Contract

Inputs are parameters. The returned value is the output. Type hints improve readability and tool support, but Python does not automatically enforce them at runtime.

Project Connection

A control helper may accept the desired and measured altitude, then return a limited correction. Keep calculation separate from message publication when possible.

Modules and Imports

Separate the project into focused, reusable files

```
# trajectory.py
def tracking_error(desired, measured):
    ...

# orbit_controller.py
from .trajectory import tracking_error
```

Module

A Python file containing related functions, classes, and constants.

Package

A directory that organizes related modules under one importable namespace.

Design Rule

Keep trajectory mathematics, ROS 2 node logic, configuration handling, and tests in

1 Python Refresher for Robotics
- Core Language Foundations
- Functions and Program Structure
- Object-Oriented Python for Robotics
- Errors, Files, and Configuration
- Python Environments and Dependencies
- NumPy and Basic Robotics Data Processing
- Logging, Debugging, and Integration

2 End-to-End Object-Oriented Python for Robotics

Why Object-Oriented Programming?

Keep related state and behavior together

Procedural view: data + separate functions

Object-oriented view: state + behavior inside one object

Class, Object, Attributes, and Methods

The blueprint and the individual instance

```
class DroneState:
    def __init__(self, north_m, east_m, down_m):
        self.north_m = north_m
        self.east_m = east_m
        self.down_m = down_m

    def altitude_m(self):
        return -self.down_m

state = DroneState(2.0, -1.0, -4.0)
print(state.altitude_m())
```

Vocabulary

Class: blueprint. **Object**: one instance. **Attributes**: stored state. **Methods**: operations performed by the object.

Constructor and `self`

Initialize every object in a valid and understandable state

```
class Detection:
    def __init__(self, label, confidence, center_px):
        self.label = label
        self.confidence = confidence
        self.center_px = center_px
```

`__init__`

Runs during object construction and stores the initial state.

`self`

Refers to the specific object whose method is currently running.

Engineering Habit

Validate constructor inputs when an invalid object would make later behavior ambiguous or unsafe.

Encapsulation and Clear Responsibilities

Expose useful behavior and keep implementation details local

```
class AltitudeMonitor:
    def __init__(self, limit_m):
        self._limit_m = float(limit_m)

    def within_limit(self, altitude_m):
        return 0.0 <= altitude_m <= self._limit_m
```

Python Convention

A leading underscore such as `_limit_m` means "internal implementation detail." It is a convention, not a security boundary.

Project Connection

A class should have one coherent responsibility: mission state, telemetry filtering, detection processing, or ROS 2 communication.

Inheritance in ROS 2 Nodes

Extend an existing class instead of rebuilding its capabilities

```
from rclpy.node import Node

class OrbitController(Node):
    def __init__(self):
        super().__init__("orbit_controller")
        self.timer = self.create_timer(0.05, self.tick)

    def tick(self):
        self.get_logger().info("Controller active")
```

Meaning

`OrbitController` is a specialized `Node`. Through inheritance it receives ROS 2 node capabilities, while adding project-specific behavior.

Design Restraint

Prefer composition for unrelated capabilities. Use inheritance when the "is a" relationship is clear and supported by the framework.

State Machines as Objects

Make mission phases explicit and mutually understandable

```
from enum import Enum, auto

class State(Enum):
    WAITING = auto()
    CLIMB = auto()
    ORBIT = auto()
    LANDING = auto()

current_state = State.WAITING
```

Why This Is Better Than Magic Numbers

`State.ORBIT` communicates intent. A mission controller can store one current state and execute only the behavior associated with that phase.

Internship Connection

1 Python Refresher for Robotics
- Core Language Foundations
- Functions and Program Structure
- Object-Oriented Python for Robotics
- Errors, Files, and Configuration
- Python Environments and Dependencies
- NumPy and Basic Robotics Data Processing
- Logging, Debugging, and Integration

2 End-to-End Object-Oriented Python for Robotics

Syntax Errors and Exceptions

Different failures occur at different moments

Syntax Error

Python cannot parse the program. Typical causes include missing colons, parentheses, or incorrect indentation.

Exception

The program starts, but a runtime operation fails, such as opening a missing file or converting invalid text to a number.

Debugging Order

Read the final exception type and message, then inspect the traceback from the point where your code first appears.

Handling Expected Exceptions

Recover only when the program has a meaningful response

```
try:
    altitude_m = float(raw_altitude)
except ValueError as exc:
    logger.error("Invalid altitude: %s", exc)
    altitude_m = None
```

Good Practice

Catch specific exceptions, preserve useful evidence, and choose a clear fallback such as rejecting configuration, skipping one record, or entering a passive state.

Avoid

Do not use a silent broad `except:`. It can hide programming defects and leave a robotics process running with an unknown state.

Raising Exceptions During Validation

Reject invalid assumptions close to their source

```
def validate_altitude(altitude_m, max_altitude_m):
    if not 0.5 <= altitude_m <= max_altitude_m:
        raise ValueError("altitude_m outside limits")
```

Project Connection

Mission parameters should be checked before commands are published. A clear `ValueError` is easier to diagnose than incorrect motion caused by an invalid value.

Principle

Exceptions report exceptional conditions. Normal mission choices should remain explicit conditions and state transitions.

Reading Configuration Files

Separate adjustable settings from program logic

```
import json
from pathlib import Path

config_path = Path("config/mission.json")
with config_path.open("r", encoding="utf-8") as file:
    config = json.load(file)

radius_m = float(config["radius_m"])
```

Why `with` Matters

The context manager closes the file correctly, including when an exception occurs.

Robotics Use

Store target names, thresholds, paths, or non-ROS utility settings outside the source

Writing Telemetry or Test Evidence

Persist results in a format that can be inspected later

```
import csv

with open("telemetry.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["time_s", "altitude_m"])
    writer.writerow([0.0, 1.2])
```

Use Cases

Record test measurements, tracking error, detection confidence, or mission-state transitions for later analysis.

System Design Note

High-rate robotics logging needs deliberate buffering and storage design. This simple example teaches the file-handling pattern, not a production telemetry architecture.

1 Python Refresher for Robotics
- Core Language Foundations
- Functions and Program Structure
- Object-Oriented Python for Robotics
- Errors, Files, and Configuration
- Python Environments and Dependencies
- NumPy and Basic Robotics Data Processing
- Logging, Debugging, and Integration

2 End-to-End Object-Oriented Python for Robotics

Virtual Environments

One isolated Python toolbox for one project

System Python $\neq$ project virtual environment

Why

Different projects may require different package versions. Isolation reduces conflicts and protects the operating-system Python environment.

Mental Model

The environment is a project-specific toolbox containing a Python interpreter context and installed packages.

Reproducibility

The environment directory is normally recreated from documented commands and dependency files, not copied between computers or committed to Git.

Create and Use a Virtual Environment

A repeatable Ubuntu workflow

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python --version
python -m pip --version

deactivate
```

How to Verify

After activation, `python` and `python -m pip` should refer to the environment. Verify rather than assuming from the prompt alone.

ROS 2 Compatibility

Some ROS 2 Python packages depend on the system Python environment. Follow the

Install Packages with `pip`

Install into the interpreter you intentionally selected

```
python -m pip install numpy
python -m pip install -r requirements.txt
python -m pip list
```

Why `python -m pip`?

It runs `pip` through the selected Python interpreter, reducing ambiguity when several Python installations exist.

Project Practice

Document required versions in the repository. A dependency file supports repeatable installation on the second computer and by the internship students.

Avoid

Do not install project packages into the system Python with `sudo pip`. Use the documented environment and package-management strategy.

1 Python Refresher for Robotics
- Core Language Foundations
- Functions and Program Structure
- Object-Oriented Python for Robotics
- Errors, Files, and Configuration
- Python Environments and Dependencies
- NumPy and Basic Robotics Data Processing
- Logging, Debugging, and Integration

2 End-to-End Object-Oriented Python for Robotics

Why NumPy?

Robotics data often arrives as vectors, matrices, and images

```
import numpy as np

position_m = np.array([2.0, -1.0, 4.0])
velocity_mps = np.array([0.2, 0.0, -0.1])
```

Array Mental Model

A NumPy array is a structured grid of mostly same-type values. A vector is one-dimensional, a table is two-dimensional, and an image is commonly a two- or three-dimensional array.

Robotics Uses

Positions, velocities, point collections, camera frames, bounding boxes, masks, transformations, and batches of sensor samples.

Array Shape, Data Type, and Indexing

Understand the structure before processing the values

```
image = np.zeros((480, 640, 3), dtype=np.uint8)
print(image.shape)   # (height, width, channels)
print(image.dtype)   # uint8

pixel = image[120, 300]
```

shape

The size along each axis. Never assume image dimensions without checking.

dtype

The numeric representation. It affects range, precision, memory, and arithmetic behavior.

Vectorized Calculations

Express operations on complete arrays

```
measurements_m = np.array([2.4, 2.1, 1.9, 2.2])
mean_m = measurements_m.mean()
minimum_m = measurements_m.min()
valid = measurements_m > 0.0
filtered = measurements_m[valid]
```

What Happened?

NumPy calculated statistics and applied one condition to every measurement. The Boolean mask selected only valid values.

Engineering Rule

Vectorization can improve clarity and performance, but first verify array shape, units, coordinate frame, and data type.

Distance and Tracking Error

A small numerical pattern used throughout robotics

```
import numpy as np

desired_m = np.array([5.0, 0.0, -4.0])
measured_m = np.array([4.7, 0.2, -3.9])
error_vector_m = desired_m - measured_m
tracking_error_m = np.linalg.norm(error_vector_m)
```

Interpretation

Subtract positions to obtain an error vector. Its Euclidean norm gives one scalar distance between desired and measured positions.

Coordinate Discipline

Only subtract values expressed in compatible units and coordinate frames. NED and ENU vectors cannot be mixed without an explicit transformation.

Processing Detection Results

Filter observations before they enter the wider system

```
detections = [
    {"label": "car", "confidence": 0.88},
    {"label": "person", "confidence": 0.41},
]

accepted = [d for d in detections
            if d["confidence"] >= 0.60]
```

Pipeline Role

Python converts model output into structured observations: labels, confidence values, bounding boxes, timestamps, and frame identifiers.

Control Boundary

An accepted detection is still an observation. It should not directly command the

1 Python Refresher for Robotics
- Core Language Foundations
- Functions and Program Structure
- Object-Oriented Python for Robotics
- Errors, Files, and Configuration
- Python Environments and Dependencies
- NumPy and Basic Robotics Data Processing
- Logging, Debugging, and Integration

2 End-to-End Object-Oriented Python for Robotics

Logging Instead of Scattered Prints

Record what the program did and why

```
import logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

logger.info("Mission state: %s", "ORBIT")
logger.warning("No camera frame received")
logger.error("Configuration file is invalid")
```

Severity Levels

Use `DEBUG` for detailed diagnosis, `INFO` for normal milestones, `WARNING` for abnormal but recoverable situations, and `ERROR` for failed operations.

Logging Inside a ROS 2 Node

Use the framework logger for node-level evidence

```
self.get_logger().info("Mission ready")
self.get_logger().warning("Waiting for odometry")
self.get_logger().error("Invalid mission parameter")
```

Useful Log Content

Include the component, mission state, relevant value with units, and expected response. Avoid flooding the terminal from high-rate callbacks.

Project Connection

The orbit controller logs readiness, state transitions, and tracking error while ROS 2 manages node-specific output.

A Disciplined Debugging Workflow

Observe, isolate, test, and verify

1. Reproduce the problem with the exact command and environment.
2. Read the complete traceback or ROS 2 log message.
3. Identify the first relevant file, line, variable, topic, or callback.
4. Inspect values, types, array shapes, paths, and active environment.
5. Test one hypothesis with the smallest possible change.
6. Rerun the original scenario and record the verified result.

Avoid Random Repair

Repeated reinstallation, broad exception handling, and unrelated code changes destroy evidence and make the defect harder to isolate.

From Python Function to ROS 2 Callback

The same fundamentals appear inside event-driven robotics code

```
def odometry_callback(self, msg):
    position = tuple(float(v) for v in msg.position)
    self.latest_position = position
    self.get_logger().debug(
        "Position NED: %s", position)
```

Concept Mapping

Function becomes callback, **object attribute** stores state, **tuple** represents position, **loop/comprehension** converts values, and **logging** supports diagnosis.

ROS 2 Reminder

The callback runs when a message arrives. Keep it predictable, non-blocking, and focused on one responsibility.

A Small Telemetry Publisher Pattern

Compute first, validate second, publish third

```
def publish_altitude(self, down_m):
    altitude_m = -float(down_m)
    if altitude_m < 0.0:
        self.get_logger().warning("Invalid altitude")
        return

    message = Float32()
    message.data = altitude_m
    self.altitude_publisher.publish(message)
```

System Pattern

Receive input, convert representation, validate assumptions, build a typed message, publish, and log unusual conditions.

How the Concepts Appear in the Internship Code

The refresher prepares you to read a complete robotics node

- **Variables and dictionaries**: mission parameters and limits.
- **Tuples and lists**: positions, setpoints, and message fields.
- **Conditions and state enumeration**: mission transitions.
- **Functions and methods**: validation, timing, commands, and callbacks.
- **Class inheritance**: the controller extends a ROS 2 `Node`.
- **Exceptions**: invalid parameters stop initialization clearly.
- **Imports and modules**: trajectory mathematics remain separate.
- **Logging**: readiness, state changes, and tracking error are observable.

Reading Strategy

Do not read the file as one long program. Identify the class state, initialization, communication interfaces, callbacks, helper methods, and repeating timer logic.

Knowledge Check

You are ready to continue when you can explain these ideas

- Which Python container best represents one position, a measurement history, and named configuration?
- Why should a distance function be separated from ROS 2 publication?
- What is the difference between a class and an object?
- Why does a ROS 2 Python node inherit from `Node`?
- When should an exception be caught, and when should it stop the program?
- Why use a virtual environment and `python -m pip`?
- What do NumPy `shape` and `dtype` tell us?
- Why is an AI detection not a direct flight-control command?

Python-to-Robotics Roadmap

Foundations become components of the complete project

Python fundamentals → structured functions and classes → NumPy data processing

→ ROS 2 nodes and callbacks → Gazebo/PX4 integration → AI observations

Python-to-Robotics Roadmap

Foundations become components of the complete project

Python fundamentals → structured functions and classes → NumPy data processing

→ ROS 2 nodes and callbacks → Gazebo/PX4 integration → AI observations

Next Step

The following ROS 2 tutorials will reuse these concepts to create packages, define nodes, exchange typed messages, process callbacks, and connect Python logic to the simulated drone system.

1 Python Refresher for Robotics

2 End-to-End Object-Oriented Python for Robotics
- Object-Oriented Python for Robotics
- References and Further Practice

1 Python Refresher for Robotics

2 End-to-End Object-Oriented Python for Robotics
- Object-Oriented Python for Robotics
- References and Further Practice

Object-Oriented Python: End-to-End Goal

Build one small robotics component progressively

From raw telemetry to a reusable drone monitor object

Object-Oriented Python: End-to-End Goal

Build one small robotics component progressively

From raw telemetry to a reusable drone monitor object

Store State

Position, battery level, flight mode, and update time.

Apply Behavior

Update telemetry, calculate altitude, and validate values.

Integrate

Use the object inside a Python ROS 2 node.

Learning Scope

The objective is practical OOP for internship code, not every advanced feature of the Python object model.

Why Move Beyond Separate Variables?

Related data becomes difficult to manage as the system grows

Procedural Starting Point

A small script may store altitude, battery, mode, and position in separate variables and pass them repeatedly between functions.

Why Move Beyond Separate Variables?

Related data becomes difficult to manage as the system grows

Procedural Starting Point

A small script may store altitude, battery, mode, and position in separate variables and pass them repeatedly between functions.

The Scaling Problem

With several drones, sensors, callbacks, and mission phases, it becomes easy to mix values belonging to different entities or update only part of the state.

OOP Response

A class defines one meaningful type. Each object keeps its own state and exposes the operations allowed on that state.

Class = blueprint **Object = one constructed instance**

Step 1: Define a Robotics Class

Group drone state under one clear concept

```
class DroneState:
    """Store the latest estimated state of one drone."""

    def __init__(self, drone_id):
        self.drone_id = drone_id
        self.position_ned_m = (0.0, 0.0, 0.0)
        self.battery_pct = 100.0
        self.flight_mode = "IDLE"

state = DroneState("uav_1")
```

What Python Creates

`DroneState` is the class. `state` is one object. The attributes belong to that specific object.

Step 2: Understand the Constructor and `self`

Initialize every object with a predictable state

```
def __init__(self, drone_id):
    self.drone_id = drone_id
    self.position_ned_m = (0.0, 0.0, 0.0)
    self.battery_pct = 100.0
    self.flight_mode = "IDLE"
```

`__init__`

Runs automatically after Python creates a new object.

`self`

Refers to the particular object whose method is running.

Intuition

`self.battery_pct` means "the battery value belonging to this drone state object."

Step 3: Add Methods

Behavior operates on the object's own state

```
class DroneState:
    def __init__(self, drone_id):
        self.drone_id = drone_id
        self.position_ned_m = (0.0, 0.0, 0.0)

    def update_position(self, north_m, east_m, down_m):
        self.position_ned_m = (
            float(north_m), float(east_m), float(down_m)
        )

    def altitude_m(self):
        return -self.position_ned_m[2]
```

Responsibility

The object stores NED position and provides the conversion from *Down* to positive

Step 4: Validate State Changes

Do not allow an object to accept meaningless values silently

```
def update_battery(self, battery_pct):
    battery_pct = float(battery_pct)

    if not 0.0 <= battery_pct <= 100.0:
        raise ValueError(
            "battery_pct must be between 0 and 100"
        )

    self.battery_pct = battery_pct
```

Engineering Principle

Validation belongs close to the state it protects. A clear exception prevents invalid telemetry from becoming accepted object state.

Step 5: Encapsulation by Convention

Separate the public interface from internal representation

```
class DroneState:
    def __init__(self, drone_id):
        self.drone_id = drone_id
        self._position_ned_m = (0.0, 0.0, 0.0)

    def update_position(self, north_m, east_m, down_m):
        self._position_ned_m = (
            float(north_m), float(east_m), float(down_m)
        )
```

Python Convention

A leading underscore communicates: "this attribute is an internal implementation detail." It is not a security mechanism.

Benefit

Step 6: Expose a Readable Property

Provide value-like access while keeping calculation inside the class

```
class DroneState:
    def __init__(self, drone_id):
        self.drone_id = drone_id
        self._position_ned_m = (0.0, 0.0, 0.0)

    @property
    def altitude_m(self):
        return -self._position_ned_m[2]

state = DroneState("uav_1")
print(state.altitude_m)
```

When a Property Fits

Use a property for a simple, inexpensive value derived from object state. Use a normal

Step 7: Add a Useful String Representation

Make objects easier to inspect during debugging

```
def __str__(self):
    return (
        f"{self.drone_id}: mode={self.flight_mode}, "
        f"altitude={self.altitude_m:.2f} m, "
        f"battery={self.battery_pct:.1f}%"
    )

print(state)
```

Why This Helps

`__str__` converts the object into a concise human-readable summary for terminal output, logs, and debugging.

Logging Practice

Do not print high-rate object summaries on every callback. Log intentionally and use

Step 8: Create Multiple Independent Objects

One class can represent several drones without mixing their state

```
uav_1 = DroneState("uav_1")
uav_2 = DroneState("uav_2")

uav_1.update_position(2.0, 1.0, -4.0)
uav_2.update_position(-3.0, 0.5, -2.0)

print(uav_1.altitude_m)  # 4.0
print(uav_2.altitude_m)  # 2.0
```

Core OOP Benefit

Both objects share the same class definition, but each object owns a separate set of attribute values.

Robotics Analogy

The class is the vehicle state design. Each simulated drone receives its own state

Step 9: Composition

Build a larger component from smaller focused objects

```
class BatteryMonitor:
    def __init__(self, warning_threshold_pct=25.0):
        self.warning_threshold_pct = warning_threshold_pct

    def is_low(self, battery_pct):
        return battery_pct <= self.warning_threshold_pct

class DroneMonitor:
    def __init__(self, drone_id):
        self.state = DroneState(drone_id)
        self.battery_monitor = BatteryMonitor()
```

Meaning

A `DroneMonitor` *has a* state and *has a* battery monitor. This is composition.

Step 10: Inheritance

Create a specialized type when the "is a" relationship is true

```
from rclpy.node import Node

class DroneMonitorNode(Node):
    def __init__(self):
        super().__init__("drone_monitor")
        self.monitor = DroneMonitor("uav_1")
```

Relationship

`DroneMonitorNode` *is a* ROS 2 `Node`, so inheritance is appropriate. It also *has a* `DroneMonitor`, so composition is appropriate for the project logic.

Avoid

Do not use inheritance only to reuse a few lines of code. Use it when the child genuinely satisfies the conceptual contract of the parent type.

Step 11: Connect a ROS 2 Subscription

A callback updates the composed domain object

```
class DroneMonitorNode(Node):
    def __init__(self):
        super().__init__("drone_monitor")
        self.monitor = DroneMonitor("uav_1")
        self.subscription = self.create_subscription(
            VehicleOdometry,
            "/fmu/out/vehicle_odometry",
            self.odometry_callback,
            10,
        )

    def odometry_callback(self, msg):
        self.monitor.state.update_position(*msg.position)
```

Concept Mapping

Step 12: Add Logging and Error Handling

Keep the node observable without hiding failures

```
def odometry_callback(self, msg):
    try:
        self.monitor.state.update_position(*msg.position)
    except (TypeError, ValueError) as exc:
        self.get_logger().error(
            f"Invalid odometry message: {exc}"
        )
        return

    self.get_logger().debug(str(self.monitor.state))
```

Callback Pattern

Receive data, convert it, validate it through the object interface, handle expected failures, update state, and record useful evidence.

Step 13: Keep Configuration Outside the Class

Construct objects from explicit project settings

```
config = {
    "drone_id": "uav_1",
    "battery_warning_pct": 25.0,
}

state = DroneState(config["drone_id"])
battery_monitor = BatteryMonitor(
    warning_threshold_pct=config["battery_warning_pct"]
)
```

Separation of Concerns

Configuration chooses values. The class defines behavior and validates the values it receives. Avoid hiding project-specific constants inside methods.

ROS 2 Connection

Step 14: Test the Object Without ROS 2

Domain logic should be testable independently of communication

```
def test_altitude_from_ned_position():
    state = DroneState("test_uav")
    state.update_position(1.0, 2.0, -4.5)
    assert state.altitude_m == 4.5


def test_invalid_battery_is_rejected():
    state = DroneState("test_uav")
    try:
        state.update_battery(120.0)
        assert False
    except ValueError:
        assert True
```

Testing Principle

Complete End-to-End OOP Example I

Domain classes: state and monitoring logic

```
class DroneState:
    def __init__(self, drone_id):
        self.drone_id = drone_id
        self._position_ned_m = (0.0, 0.0, 0.0)
        self.battery_pct = 100.0
        self.flight_mode = "IDLE"

    def update_position(self, north_m, east_m, down_m):
        self._position_ned_m = (
            float(north_m), float(east_m), float(down_m)
        )

    def update_battery(self, battery_pct):
        value = float(battery_pct)
        if not 0.0 <= value <= 100.0:
            raise ValueError("battery_pct outside [0, 100]")
        self.battery_pct = value

    @property
    def altitude_m(self):
        return -self._position_ned_m[2]
```

Complete End-to-End OOP Example II

Composition: evaluate the state without controlling the drone

```
class BatteryMonitor:
    def __init__(self, warning_threshold_pct=25.0):
        threshold = float(warning_threshold_pct)
        if not 0.0 <= threshold <= 100.0:
            raise ValueError("invalid warning threshold")
        self.warning_threshold_pct = threshold

    def is_low(self, battery_pct):
        return float(battery_pct) <= self.warning_threshold_pct


class DroneMonitor:
    def __init__(self, drone_id, battery_warning_pct=25.0):
        self.state = DroneState(drone_id)
        self.battery_monitor = BatteryMonitor(
            battery_warning_pct
        )

    def status(self):
        return "LOW_BATTERY" if self.battery_monitor.is_low(
            self.state.battery_pct
```

Complete End-to-End OOP Example III

ROS 2 integration: framework class plus project objects

```
class DroneMonitorNode(Node):
    def __init__(self):
        super().__init__("drone_monitor")
        self.monitor = DroneMonitor("uav_1", 25.0)
        self.subscription = self.create_subscription(
            VehicleOdometry,
            "/fmu/out/vehicle_odometry",
            self.odometry_callback,
            10,
        )

    def odometry_callback(self, msg):
        try:
            self.monitor.state.update_position(*msg.position)
        except (TypeError, ValueError) as exc:
            self.get_logger().error(
                f"Rejected odometry: {exc}"
            )
            return

        self.get_logger().info(
```

How to Read an OOP Robotics File

Do not read hundreds of lines from top to bottom immediately

1. Identify each class and write its responsibility in one sentence.
2. Find the constructor and list the object's persistent attributes.
3. Separate public behavior from internal helper methods.
4. Locate framework interfaces: publishers, subscriptions, services, timers.
5. Trace one event from callback input to object update and output.
6. Identify validation, exception handling, logging, and shutdown behavior.

Apply This to the Orbit Controller

The controller *is a* ROS 2 node. It owns mission parameters, communication interfaces, measured state, desired state, paths, and a timer. Its methods implement callbacks, commands, state transitions, and publication.

OOP Design Checklist for the Internship

Simple rules for maintainable robotics Python

- Give each class one coherent responsibility.
- Initialize objects in a valid, predictable state.
- Use explicit names with units and coordinate frames.
- Validate values at system boundaries.
- Prefer composition for collaborating capabilities.
- Use inheritance when a genuine “is a” relationship exists.
- Keep callbacks short and avoid blocking operations.
- Separate calculations from ROS 2 communication when practical.
- Log useful evidence without flooding high-rate loops.
- Test domain logic independently from the complete simulation.

OOP Knowledge Check

Explain the design, not only the syntax

- What is the difference between a class and an object?
- Why does every instance method receive `self`?
- Which state should be initialized in `__init__`?
- When should behavior be a property rather than a method?
- Why is `DroneMonitorNode` inheritance, while its `DroneMonitor` relationship is composition?
- Where should telemetry values be validated?
- Why should domain calculations be testable without ROS 2?
- How does a state-machine class improve mission clarity?

Next: errors, files, configuration, environments, and NumPy

1 Python Refresher for Robotics

2 End-to-End Object-Oriented Python for Robotics
- Object-Oriented Python for Robotics
- References and Further Practice

References and Further Reading I

Python language and program structure

Python Software Foundation, *The Python Tutorial*.
https://docs.python.org/3/tutorial/

Python Software Foundation, *Classes*.
https://docs.python.org/3/tutorial/classes.html

Python Software Foundation, *Errors and Exceptions*.
https://docs.python.org/3/tutorial/errors.html

Python Software Foundation, *Modules*.
https://docs.python.org/3/tutorial/modules.html

Python Software Foundation, *PEP 8: Style Guide for Python Code*.
https://peps.python.org/pep-0008/

References and Further Reading II

Environments, packages, data processing, and logging

Python Software Foundation, *venv: Creation of Virtual Environments*.
https://docs.python.org/3/library/venv.html

Python Packaging Authority, *pip User Guide*.
https://pip.pypa.io/en/stable/user_guide/

Python Packaging Authority, *Installing Packages*.
https://packaging.python.org/en/latest/tutorials/installing-packages/

NumPy Developers, *NumPy: The Absolute Basics for Beginners*.
https://numpy.org/doc/stable/user/absolute_beginners.html

Python Software Foundation, *Logging HOWTO*.
https://docs.python.org/3/howto/logging.html